

[Secure Exit](#)

CO3-AT3-Debugging and Optimization

[View Only](#)[← Back](#)

QUESTIONS

Q1ATM Withdrawal – Exception
Handling Debugging

10 marks

**Q2**Hospital Registration –
NullPointerException
Debugging

10 marks

**Q3**E-Commerce Product Search –
Array Exception Debugging

10 marks



Q5 Railway Reservation – Synchronization Debugging

10 marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.



SIMATS Online Programming Lab

JAVA PROGRAMMINGSUBIKSHA U
192424135RA**Q5**Railway Reservation –
Synchronization Debugging

```
void bookSeat(String customer) {
```

```
    if (seats > 0) {
```

10 marks

5/5 answered

```
System.out.println(customer
+ " is booking a seat");

try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Thread interrupted");
}

seats--;

System.out.println(customer
+ " booking successful");

} else {
    System.out.println(customer
+ " - No seat available");
}
}

class Customer extends Thread {

    Reservation reservation;
    String customerName;

    Customer(Reservation reservation, String customerName) {
        this.reservation = reservation;
        this.customerName = customerName;
    }
}
```

```
public void run() {  
    reservation.bookSeat(customerName);  
}  
}  
  
class RailwayReservation {  
  
    public static void main(String[] args) {  
  
        Reservation reservation = new Reservation();  
  
        Customer c1 =  
            new Customer(reservation, "Customer 1");  
  
        Customer c2 =  
            new Customer(reservation, "Customer 2");  
  
        c1.start();  
        c2.start();  
    }  
}
```

The corrected program must protect the shared seat resource using synchronization.

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 class Reservation{  
2     int seat = 1;  
3
```

Test Input

```

4      synchronized void bookSeat(String customer){
5          if(seat>0){
6              System.out.println(customer + " is book
7
8              try{
9                  Thread.sleep(100);
10             }catch(InterruptedException e){
11                 System.out.println("Thread interrup
12             }
13             seat--;
14
15             System.out.println(customer + "booking
16         }else{
17             System.out.println(customer + " NO seat
18         }
19     }
20     public static void main(String[]args){
21         Reservation reservation = new Reservation()
22         Customer c1 = new Customer(reservation,"Cus
23         Customer c2 = new Customer(reservation, "Cu
24         c1.start();
25         c2.start();
26
27     }
28 }
29 class Customer extends Thread{
30     Reservation reservation;
31     String customerName;
32
33     Customer(Reservation reservation, String custom
34         this.reservation = reservation;
35         this.customerName = customerName;
36     }

```

stdin input (optional)

Output

Customer 1: is booking
a seat
Customer 1:booking
successful
Customer 2: NO seat
available

⌚ Pending manual review

```
37     public void run() {  
38         reservation.bookSeat(customerName);  
39     }  
40 }  
41 }
```

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.